LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 4

STUKDAT



Disusun oleh:

[MUHAMMAD NAUFAL AKMAL PRABOWO]

109082500186

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul X : (https://github.com/naufalakmall/struktur-data.git)

1. Guided

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
```

```
            insertFirst(L, P);
        } else {
            address Q = L.first;
            while (Q->next != NULL) {
                Q = Q->next;
            }
            Q->next = P;
        }
    }

    void insertAfter(List &L, address P, address Prec) {
        if (Prec != NULL) {
            P->next = Prec->next;
            Prec->next = P;
        }
    }

    void deleteFirst(List &L, address &P) {
        if (L.first != NULL) {
            P = L.first;
            L.first = P->next;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        if (L.first != NULL) {
            if (L.first->next == NULL) {
                deleteFirst(L, P);
            } else {
                address Q = L.first;
                while (Q->next->next != NULL) {
                    Q = Q->next;
                }
                P = Q->next;
                Q->next = NULL;
            }
        }
    }

    void deleteAfter(List &L, address &P, address Prec) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;
            Prec->next = P->next;
            P->next = NULL;
        }
    }
```

```
void printInfo(List L) {
    address P = L.first;
    while (P != NULL) {
        cout << P->info << ' ';
        P = P->next;
    }
    cout << '\n';
}

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

2. **Soal 1**

Source Code

```
#include <iostream>
#include <string>

using namespace std;

struct Node {
    string nama;
    Node* next;
};
```

```
Node* head = nullptr;

void tambahNasabah(string namaBaru) {
    Node* newNode = new Node();
    newNode->nama = namaBaru;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

void layaniNasabah() {
    if (head == nullptr) {
        cout << "Antrean kosong." << endl;
        return;
    }

    Node* temp = head;
    cout << "Melayani: " << temp->nama << endl;
    head = head->next;
    delete temp;
}

void tampilkanAntrean() {
    if (head == nullptr) {
        cout << "Antrean: (kosong)" << endl;
        return;
    }

    cout << "Antrean: ";
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->nama;
        if (temp->next != nullptr) {
            cout << " -> ";
        }
        temp = temp->next;
    }
    cout << endl;
}
```

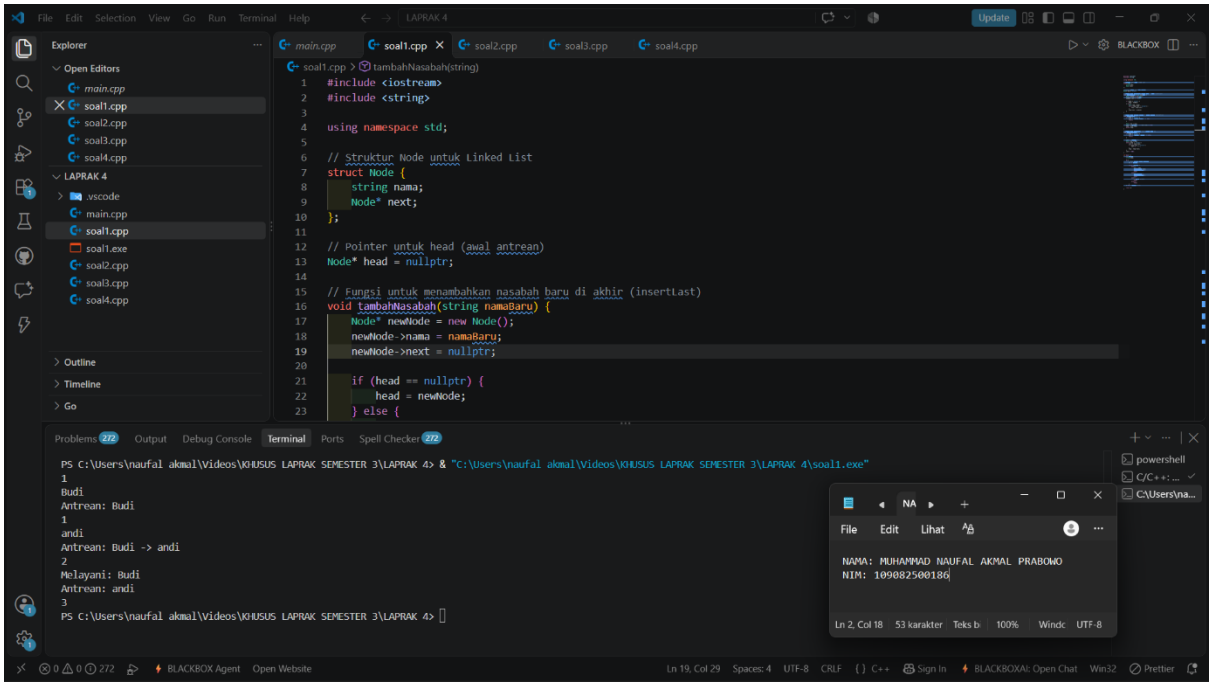
```
int main() {
    int pilihan;
    string nama;

    do {
        cin >> pilihan;

        switch (pilihan) {
            case 1:
                cin >> nama;
                tambahNasabah(nama);
                tampilkanAntrean();
                break;
            case 2:
                layaniNasabah();
                tampilkanAntrean();
                break;
            case 3:
                // Keluar program
                break;
            default:
                break;
        }
    } while (pilihan != 3);

    return 0;
}
```

Screenshoot program :



Deskripsi program

Kode C++ di atas mengimplementasikan sistem antrean nasabah sederhana menggunakan struktur data Linked List yang menerapkan konsep FIFO (First In, First Out). Program ini menyediakan fungsi untuk menambah nasabah baru di akhir antrean (tambahNasabah), melayani serta menghapus nasabah di posisi terdepan (layaniNasabah), dan menampilkan daftar keseluruhan antrean saat ini (tampilkanAntrean), yang dikendalikan melalui menu interaktif berbasis perulangan do-while di dalam fungsi main.

3. **Soal 2**

Source Code

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* head = nullptr;

pembentukan list awal
void insertLast(int nilai) {
    Node* newNode = new Node();
    newNode->data = nilai;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

dicari (insertAfter)
```

```
void insertAfter(int cariNilai, int nilaiBaru) {
    Node* temp = head;

    while (temp != nullptr && temp->data != cariNilai) {
        temp = temp->next;
    }

    if (temp != nullptr) {
        Node* newNode = new Node();
        newNode->data = nilaiBaru;
        newNode->next = temp->next;
        temp->next = newNode;
    }
}

void tampilkanList() {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr) {
            cout << " -> ";
        }
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    int n, nilai, cariNilai, nilaiBaru;

   oleh N buah bilangan bulat
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> nilai;
        insertLast(nilai);
    }


    cin >> cariNilai;

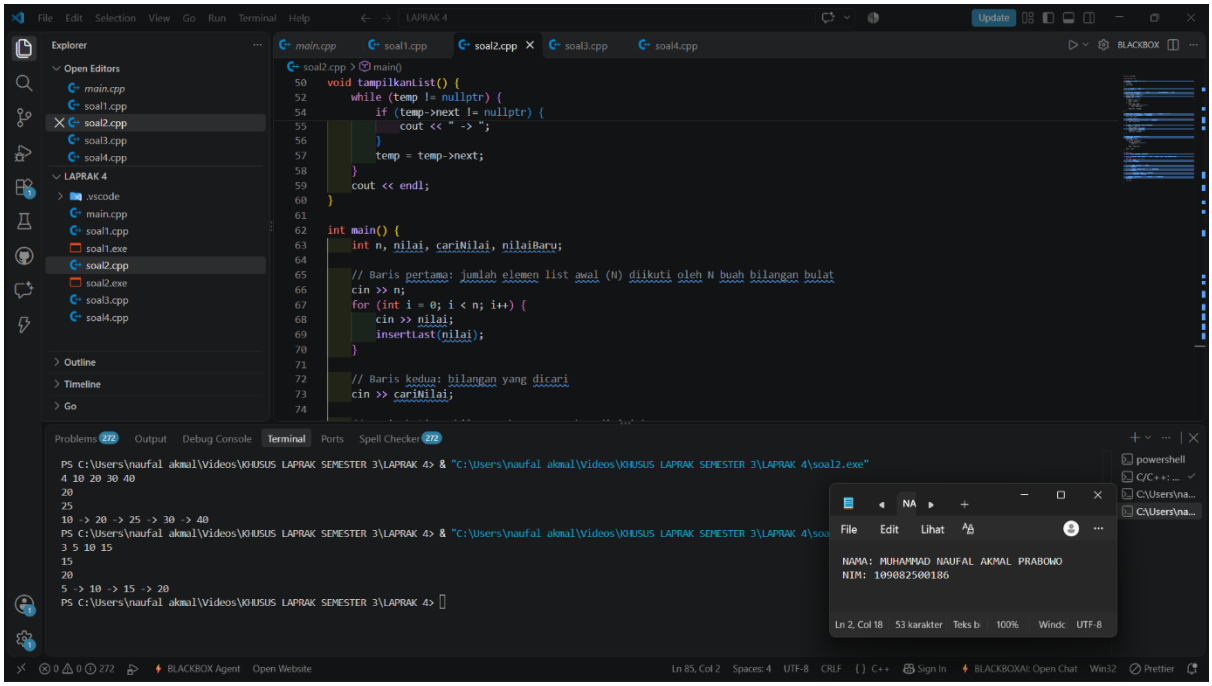

    cin >> nilaiBaru;


    insertAfter(cariNilai, nilaiBaru);

   penyisipan
    tampilkanList();
```

```
    return 0;
}
```

Screenshoot program



Deskripsi program

Kode C++ di atas mengimplementasikan operasi penyisipan data pada struktur data Single Linked List dengan menambahkan fungsi sisip setelah nilai tertentu (insertAfter). Program ini menerima masukan berupa jumlah elemen awal beserta nilai-nilainya untuk membentuk linked list menggunakan fungsi insertLast, lalu meminta nilai acuan yang ingin dicari beserta nilai baru yang akan disisipkan tepat setelah nilai acuan tersebut, sebelum akhirnya menampilkan seluruh isi elemen linked list yang telah diperbarui melalui fungsi tampilkanList.

4. Soal 3

Source Code

```
#include <iostream>

using namespace std;

struct Node {
    int data;
```

```
    Node* next;
};

Node* head = nullptr;

void insertLast(int nilai) {
    Node* newNode = new Node();
    newNode->data = nilai;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

int hitungJumlahTotal() {
    int sum = 0;
    Node* temp = head;

    while (temp != nullptr) {
        sum += temp->data;
        temp = temp->next;
    }

    return sum;
}

int main() {
    int n, nilai;

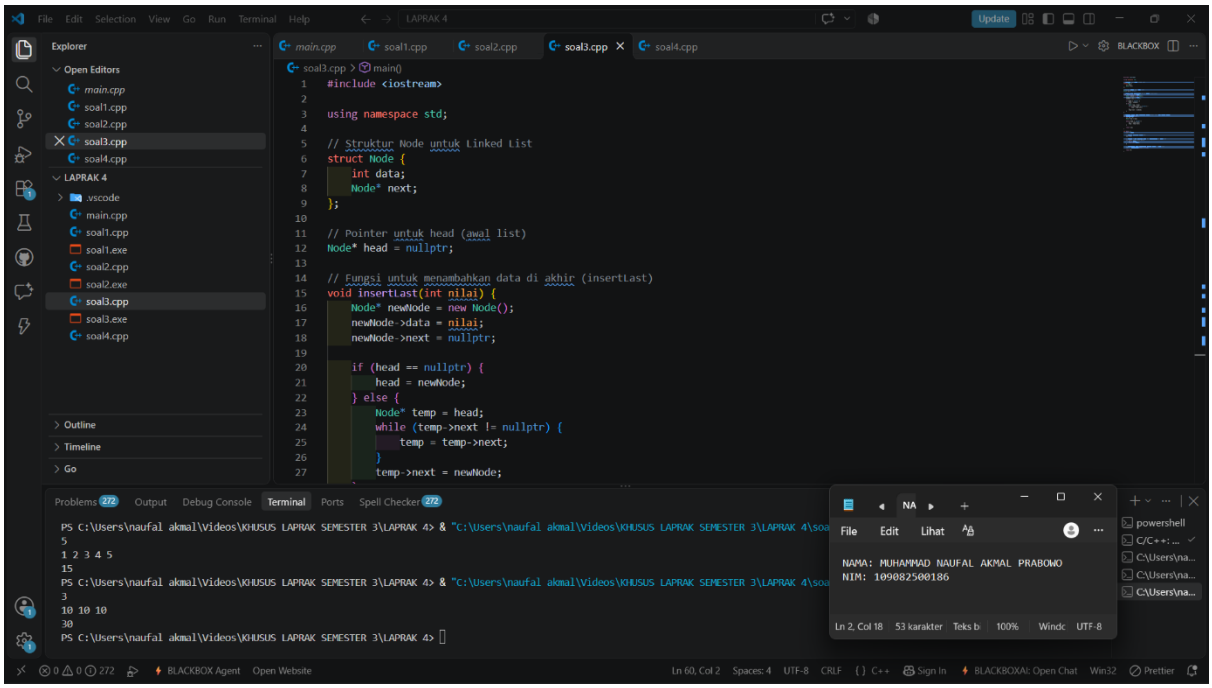
    cin >> n;

    for (int i = 0; i < n; i++) {
        cin >> nilai;
        insertLast(nilai);
    }

    cout << hitungJumlahTotal() << endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Kode C++ di atas mengimplementasikan penjumlahan seluruh nilai elemen yang tersimpan di dalam struktur data Single Linked List. Program ini membaca masukan jumlah data beserta nilainya untuk membentuk linked list melalui fungsi insertLast, lalu melakukan iterasi penelusuran menggunakan fungsi hitungJumlahTotal untuk mengakumulasi total keseluruhan nilai sebelum menampilkan hasilnya ke layar.

5. Soal 4

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Buku {
    string judul;
    int tahun;
    Buku* next;
};

Buku* head = nullptr;

void tambahBuku(string judul, int tahun) {
    Buku* baru = new Buku;
    baru->judul = judul;
```

```
        baru->tahun = tahun;
        baru->next = nullptr;

        if (head == nullptr) {
            head = baru;
        } else {
            Buku* bantu = head;
            while (bantu->next != nullptr) {
                bantu = bantu->next;
            }
            bantu->next = baru;
        }
    }

    void deleteLast() {
        if (head == nullptr) {
            return;
        }

        if (head->next == nullptr) {
            delete head;
            head = nullptr;
            return;
        }

        Buku* bantu = head;

        while (bantu->next->next != nullptr) {
            bantu = bantu->next;
        }

        delete bantu->next;
        bantu->next = nullptr;
    }

    void tampilkanBuku() {
        cout << "Daftar Buku:" << endl;

        Buku* bantu = head;

        while (bantu != nullptr) {
            cout << bantu->judul << " ("
                 << bantu->tahun << ")" << endl;
            bantu = bantu->next;
        }
    }

    int main() {
```

```
        int N;
        string judul;
        int tahun;

        cin >> N;
        cin.ignore();

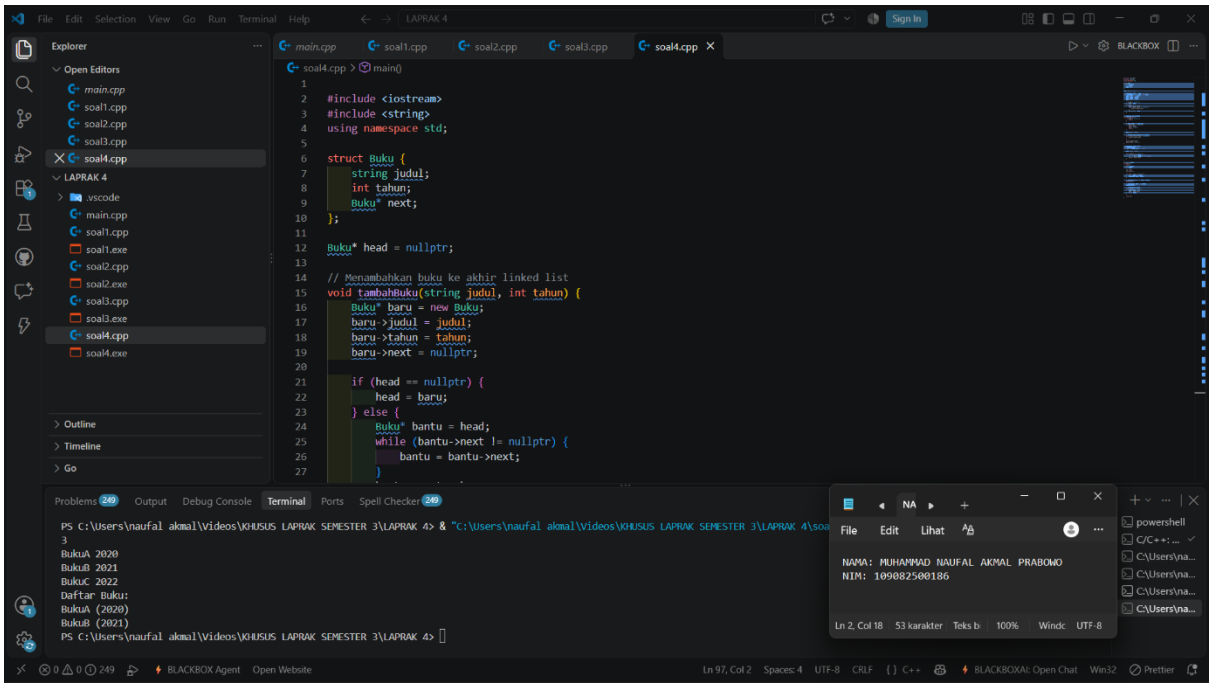
        for (int i = 0; i < N; i++) {
            cin >> judul >> tahun;
            tambahBuku(judul, tahun);
        }

        deleteLast();
        tampilkanBuku();

        while (head != nullptr) {
            Buku* hapus = head;
            head = head->next;
            delete hapus;
        }

        return 0;
    }
```

Screenshoot program



Deskripsi program

Kode C++ di atas mengimplementasikan manajemen data buku menggunakan struktur data Single Linked List yang dilengkapi dengan fitur penambahan dan penghapusan elemen pada bagian akhir. Program ini membaca masukan data buku untuk dimasukkan melalui fungsi tambahBuku, menghapus buku terakhir menggunakan fungsi deleteLast, menampilkan daftar buku yang tersisa dengan fungsi tampilkanBuku, serta melakukan dealokasi memori secara menyeluruh di akhir program.